



HASHDIT

HashDit

Smart Contract Security

ListaDao MoolahVaultManager Update Security Audit Report

ListaDao

01 June 2026 – 04 June 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	01 June 2026 — 04 Jun 2026
Overall Risk	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/183
Branch	fix/moolah-vault-manager-remove-market

Smart Contract List

No.	Contract Name	Verdict	Details
1	MoolahVaultManager.sol	Informational	[I01]
			[I02]
			[I03]
			[I04]
			[I05]
			[I06]
			[I07]

Fix Review

Bug 1	
Contract	MoolahVaultManager.sol

Severity

Low

Description

supplyQueue could legitimately exclude a market that still sat in withdrawQueue (e.g. allocator narrowed the supply queue). The old code sized newSupplyQueue as length - 1 and panicked with out-of-bounds before the "Not in supply queue" check could ever fire. The fix records supplyIdx as a sentinel; setSupplyQueue is now rebuilt only when the target is actually in supplyQueue. Only withdrawQueue is required to contain the target — otherwise the call reverts with "Not in withdraw queue".

Analysis

The withdrawal and supply queues are independent. The withdrawQueue contains every enabled market (a market is auto-added on _setCap and can't be dropped while it holds supply). The supplyQueue is freely-curated by the ALLOCATOR role and MoolahVault.setSupplyQueue only requires that each included market has cap != 0; it does not require every market to be included. So an ALLOCATOR can legitimately narrow the supply queue to exclude a market that's still live in the withdraw queue

JavaScript

```
// reallocate all assets to another markets
uint256 supplyIdx = type(uint256).max;
MarketAllocation[] memory allocations = new MarketAllocation[] (2);
allocations[0] = MarketAllocation({ marketParams: marketParams,
assets: 0 });
uint256 supplyQueueLength = IMoolahVault(vault).supplyQueueLength();
Id[] memory newSupplyQueue = new Id[] (supplyQueueLength - 1);
```

Description

Scenario:

1. Market M1 is enabled, has a cap, and the vault holds supply in it. Both queues contain M1.
2. The allocator decides to stop routing new deposits into M1 (winding it down) and calls setSupplyQueue([M2, M3]), omitting M1. M1 remains in withdrawQueue because it still has supplyShares > 0.
3. The bot calls removeMarketFromVault(vault, M1). The original code unconditionally built newSupplyQueue = new Id[] (supplyQueueLength - 1) = size-1 array, then copied every supplyQueue entry that isn't M1. Since M1 isn't there, both M2 and M3 get copied → writing 2 elements into a size-1 array → array out-of-bounds.
4. The intended "Not in supply queue" guard sat after this write, so it never ran. Removal of M1 is bricked, unless the market is added back to the contract

Status	Fixed, Now removes the j index loop which represented the index holding the non-removed markets in the original code. Now, solely relies on the supplyIdx index, wherein if the supplyIdx is not equal to uint256.max, i.e. if the target removed market id is still within the supplyQueue, do we rebuilt the newSupplyQueue and set the new supplyQueue.
---------------	--

Bug 2

Contract	MoolahVaultManager.sol
-----------------	------------------------

Severity	Low
-----------------	------------

Description	Description After a bad-debt socialization a vault can hold supplyShares > 0 with expectedSupplyAssets == 0. The old code skipped reallocate (gated on vaultSupplyAssets > 0) and then updateWithdrawQueue reverted with InvalidMarketRemovalNonZeroSupply. The fix invokes setMarketRemoval after setCap(0) whenever residual supplyShares remain, so cleanup completes in the same tx. The stale Moolah shares are intentionally left behind — their asset value is 0 and virtual-share dilution keeps any future claim economically ≈ 0, so no assets are lost. Analysis After socialization, a vault can hold supplyShares > 0 while expectedSupplyAssets == 0, because shares survive but assets-per-share collapses (Moolah.sol:578). <pre> JavaScript market[id].totalBorrowAssets -= badDebtAssets.toUint128(); market[id].totalSupplyAssets -= badDebtAssets.toUint128(); market[id].totalBorrowShares -= badDebtShares.toUint128(); position[id][borrower].borrowShares = 0; </pre> Scenario: 1. Market M has borrowers; the vault supplied into it, so position(M, vault).supplyShares = S_vault > 0. 2. A large borrower goes underwater and is liquidated down to collateral == 0. Moolah socializes the loss: totalSupplyAssets -= badDebtAssets. If the bad debt wipes out essentially all of the market's supply, totalSupplyAssets → -0 while totalSupplyShares is unchanged. 3. Now the vault's expectedSupplyAssets = toAssetsDown(S_vault, -0, totalShares) = 0, but S_vault is still nonzero.
--------------------	--

4. The bot calls `removeMarketFromVault(vault, M)`:

- `vaultSupplyAssets = 0` → no top-up.
- Original code gated `reallocate` on `vaultSupplyAssets > 0`, so it skipped `reallocate`,
- `setCap(M, 0)`.
- Original code did not call `setMarketRemoval`.
- `updateWithdrawQueue` then tries to drop `M`. Inside the vault: `cap == 0`, but `MOOLAH.position(M, vault).supplyShares != 0` (still `S_vault`) and `config[M].removableAt == 0` (never set) → revert `InvalidMarketRemovalNonZeroSupply`.

So the market is permanently stuck in the vault even though it has no assets, no cap, but can't be dropped because of leftover worthless shares.

Status

Fixed, still retains the `vaultSupplyAssets > 0` check for reallocation, but now calls `setMarketRemoval` i.e. the `MoolahVaultManager.sol` will now have the `CURATOR` role to allow `updateWithdrawQueue` to automatically remove the targeted market from the withdrawal queue.

Bug 3

Contract

`MoolahVaultManager.sol`

Severity

Low

Description

Description

When the vault-side deficit (`supplyAssets`) is smaller than the market's `minLoan`, calling `MOOLAH.supply(supplyAssets + 1)` reverts with `"remain supply too low"`. Even after raising the top-up to `minLoan` exactly, `Moolah.supply` mints with `toSharesDown` and `_checkSupplyAssets` evaluates with `toAssetsDown`, so after accrued interest shifts the share price the resulting position can be valued at `minLoan - 1`. The fix switches to a by-shares supply: `shares = toSharesUp(target) + 1` where `target = max(supplyAssets+1, minLoan)`, so the post-supply position deterministically clears the threshold under any share-price drift. Any surplus stays as vaultManager's `supplyShares` and the bot can drain it afterwards via `withdrawFromMoolah`. The `maxSupplyValue` check uses the actual supplied amount.

Analysis

The bug occurs when the deficit is small (`< minLoan`):

1. `supply(supplyAssets + 1)`: the manager becomes a supplier with a position `supplyAssets+1`, which is below `minLoan`. Moolah's `_checkSupplyAssets(Moolah.sol:855-859)` requires every nonzero supplier position to be $\geq \text{minLoan}$, so it reverts `REMAIN_SUPPLY_TOO_LOW`. Any small-deficit removal fails.

2. Bot raise allocations to exactly minLoan, still by-assets: supplying by assets mints shares with toSharesDown (round down), and _checkSupplyAssets re-values the position with toAssetsDown (round down again). There is a potential rounddown
 - $\text{shares} = \text{toSharesDown}(\text{minLoan}) = \text{floor}(\text{minLoan} \cdot (S+1e6)/(A+1))$
 - $\text{valued back} = \text{toAssetsDown}(\text{shares}) = \text{floor}(\text{shares} \cdot (A+1)/(S+1e6)) \leq \text{minLoan}, \text{ can be } \text{minLoan} - 1$

When the share price isn't 1:1 (i.e. after interest accrual $A \neq S$), this double-floor drops the valued position to $\text{minLoan} - 1 < \text{minLoan} \rightarrow \text{revert again}$. So even the "supply exactly minLoan" version is non-deterministic and depends on share-price exchange rate

Fixed,

Status

The allocated shares is rounded up via $\text{sharesToSupply} = \text{toSharesUp}(\text{target}) + 1$ where $\text{target} = \max(\text{supplyAssets} + 1, \text{minLoan})$. Rounding shares up plus the +1 buffer guarantees the post-supply position values $\geq \text{target} \geq \text{minLoan}$ under any share price, so _checkSupplyAssets do not revert. The transferred assets = toAssetsUp(shares) is what the maxSupplyValue gate checks, and the surplus stays as the manager's recoverable shares (drained later via withdrawFromMoolah).

Findings

[I01] Orphaned shares from removed markets can re-appreciate and be re-imported on market re-enablement	
Contract	MoolahVaultManager.sol
Severity	Informational
Description	Description This is a continuation of the previous audit report in [M02], when <code>removeMarketFromVault</code> removes a market, the vault's stale supply shares are intentionally abandoned in Moolah's position[id][vault] while the vault deletes the market's config[id] during the withdraw queue update. The design assumes these orphaned shares remain worth ≈ 0 forever due to virtual share dilution, but this assumption only holds while the market stays inactive. The market continues to exist in Moolah independently of the vault. If other suppliers and borrowers keep the market active, interest accrual increases the assets per share ratio, since accrued interest inflates total supply assets while total supply shares grow only by the fee shares minted in <code>accrueInterest</code>. The orphaned shares therefore re-appreciate over time. It is therefore presumed that the this market will be deprecated on the Moolah side via market whitelists settings If a curator later re-enables the same market with the same Id, <code>_setCap</code> treats it as a new market and updates the vault's accounting with <code>_updateLastTotalAssets(lastTotalAssets + expectedSupplyAssets(vault))</code>. This re-imports the now nonzero orphan value into the vault's accounting in a single jump. It creates a front-running surface where a depositor who anticipates the re-enablement can deposit just before it to capture a share price step-up that rightfully belongs to existing holders. It also quietly breaks the ≈ 0 valuation assumption the removal design depends on.
Recommendation	Consider the following: 1. Documenting that ListaDao will explicitly drain or forfeit the orphaned shares when removing a market, or prevent re-enabling a market for which the vault holds a pre-existing nonzero position. 2. Alternatively, zero out the orphaned position when the market is re-added so its value cannot be re-imported into the vault's accounting.
Status	Acknowledged, no fix currently implemented. It is presumed that such markets removed will never be re-enabled and added back to the specified MoolahVault.

[I02] 0 cap markets left in the supply queue blocks all market removals

Contract MoolahVaultManager.sol

Severity Informational

Description

Calling setCap(0) on a market does not remove that market from the supply queue, so a curator can leave a market with cap == 0 sitting in supplyQueue when performing a setCap(0) call directly via the MoolahVault.

When the bot later calls removeMarketFromVault to remove a different market, the rebuilt newSupplyQueue still includes the cap-0 market. The subsequent call to setSupplyQueue then reverts with UnauthorizedMarket in MoolahVault, since markets with a zero cap are not allowed in the supply queue. As a result, every removal attempt fails and the manager cannot recover on its own. This creates a desynchronization between the manager's view of the queue and the vault's validation rules, and removals remain bricked until someone manually fixes the supply queue.

Description

JavaScript

```
function setSupplyQueue(Id[] calldata newSupplyQueue) external
onlyRole(ALLOCATOR) {
    uint256 length = newSupplyQueue.length;

    if (length > ConstantsLib.MAX_QUEUE_LENGTH) revert
    ErrorsLib.MaxQueueLengthExceeded();

    for (uint256 i; i < length; ++i) {
        if (config[newSupplyQueue[i]].cap == 0) revert
        ErrorsLib.UnauthorizedMarket(newSupplyQueue[i]);
    }

    supplyQueue = newSupplyQueue;

    emit EventsLib.SetSupplyQueue(_msgSender(), newSupplyQueue);
}
```

Recommendation

When rebuilding newSupplyQueue in removeMarketFromVault, filter out any market whose cap is 0 rather than only excluding the market being removed. Alternatively, skip a market from the supply queue whenever its cap is set to 0 so the queue never contains markets that setSupplyQueue would reject.

If not, always ensure that appropriate market removals are performed on the MoolahVault side i.e. setCap → updateSupplyQueue is called before a market removal call on the MoolahVaultManager.

Status	Acknowledged, no fix currently implemented, it is presumed that the CURATOR and ALLOCATOR role would follow through on vault side actions when performing a market removal before the removeMarketFromVault is invoked.
---------------	---

[I03] Market removal can remain blocked when maxSupplyValue < minLoanValue	
Contract	MoolahVaultManager.sol
Severity	Informational
Description	Description The original contract reverted with REMAIN_SUPPLY_TOO_LOW when the supply deficit was below Moolah's minLoan. The fix addresses this by bumping the supply target up to minLoan, so when supplyAssets + 1 < minSupply the target becomes minSupply instead. The require(getValue(marketParams, assetsToTransfer) <= maxSupplyValue) check effectively reduces to minLoanValue <= maxSupplyValue. As a result, the fix only works when maxSupplyValue ≥ minLoanValue. If maxSupplyValue is configured below minLoanValue, market removal is still blocked, only now it fails with the "Exceed max supply value" revert instead of the original integration error.
Recommendation	Consider adding a validation check in setMaxSupplyValue that rejects any value below minLoanValue, ensuring maxSupplyValue ≥ minLoanValue always holds so the removal path cannot be blocked by this configuration.
Status	Acknowledged, no fix currently implemented, it is presumed that the MANAGER role is trusted and will not perform such configurations.

[I04] "No market has enough cap" Conflates Multiple Failure Causes	
Contract	MoolahVaultManager.sol
Severity	Informational
Description	Description The old code enforced two separate preconditions with distinct, precise error messages: <pre> JavaScript // MoolahVaultManager - removeMarketFromVault (old code) require(supplyIdx != type(uint256).max, "Not in supply queue"); // Scenario ① if (vaultSupplyAssets > 0) { require(allocations[1].assets > 0, "No market has enough cap"); // Scenario ② IMoolahVault(vault).reallocate(allocations); } </pre>

The Bug 1 fix in PR #183 removes the first require, collapsing all failure cases into a single revert message:

```
JavaScript
// MoolahVaultManager - removeMarketFromVault (new code)
for (uint256 i = 0; i < supplyQueueLength; i++) {
  Id marketId = IMoolahVault(vault).supplyQueue(i);
  if (Id.unwrap(id) == Id.unwrap(marketId)) { supplyIdx = i;
continue; }
  if (_getRemainCap(IMoolahVault(vault), marketId) >=
vaultSupplyAssets && allocations[1].assets == 0) {
    allocations[1] = MarketAllocation({ marketParams:
MOOLAH.idToMarketParams(marketId), assets: type(uint256).max });
  }
}
if (vaultSupplyAssets > 0) {
  require(allocations[1].assets > 0, "No market has enough cap");
// Now covers all failure cases
IMoolahVault(vault).reallocate(allocations);
}
```

"No market has enough cap" now maps to three fundamentally different root causes:

Root Cause	Old Error	New Error
supplyQueue is empty – no candidates at all	"Not in supply queue"	"No market has enough cap"
Target market is the sole member of supplyQueue	"No market has enough cap"	"No market has enough cap"
All candidate markets have insufficient remaining cap	"No market has enough cap"	"No market has enough cap"

Add a distinct guard before the destination check to surface the most actionable failure reason first:

Recommendation

```
JavaScript
if (vaultSupplyAssets > 0) {
  require(supplyQueueLength > 0, "Supply queue is empty");
  require(allocations[1].assets > 0, "No market has enough cap");
  IMoolahVault(vault).reallocate(allocations);
}
```

Status Acknowledged, no fix currently implemented.

[I05] withdrawFromMoolah NatDoc Contradicts Actual Function Behavior

Contract MoolahVaultManager.sol

Severity Informational

Description

The NatDoc for withdrawFromMoolah states it "withdraws all supplied assets," but the function actually supports partial withdrawal via its assets and shares parameters. This documentation error was introduced by the L04 fix from the previous audit — before the fix the function only supported full withdrawal; after the fix it was extended to support partial withdrawal, but the NatDoc was never updated.

Description

JavaScript

```
// MoolahVaultManager - withdrawFromMoolah
/// @dev Withdraws all supplied assets of a market from Moolah to this
contract.
function withdrawFromMoolah(Id id, uint256 assets, uint256 shares)
external onlyRole(BOT) {
    require(assets > 0 || shares > 0, "zero assets and shares");
    MarketParams memory marketParams = MOOLAH.idToMarketParams(id);
    (uint256 assets, uint256 shares) = MOOLAH.withdraw(marketParams,
assets, shares, address(this), address(this));
    emit WithdrawnFromMoolah(id, assets, shares);
}
```

Update the NatDoc to accurately reflect the function's actual behavior:

Recommendation

JavaScript

```
/// @dev Withdraws assets or shares from this contract's Moolah supply
position to this contract.
/// @param assets The amount of assets to withdraw. Set to 0 when
withdrawing by shares.
/// @param shares The amount of shares to withdraw. Set to 0 when
withdrawing by assets.
/// @dev Supports partial withdrawal; call multiple times as market
liquidity becomes available.
```

Status Acknowledged, no fix currently implemented.

[I06] removeMarketFromVault Always Reverts When Target Market Has Duplicate Entries in supplyQueue

Contract MoolahVaultManager.sol

Severity

Informational

Description

MoolahVault's setSupplyQueue interface explicitly allows duplicate market entries in the supplyQueue. When the target market appears more than once, the rebuild logic in removeMarketFromVault is flawed: the scan loop overwrites supplyIdx on every match, retaining only the **last** occurrence index; the rebuild loop skips only that index, leaving the first occurrence in newSupplyQueue.

JavaScript

```
// MoolahVaultManager - removeMarketFromVault
// Scan loop: supplyIdx is overwritten on every match, retaining only
the last occurrence
for (uint256 i = 0; i < supplyQueueLength; i++) {
    Id marketId = IMoolahVault(vault).supplyQueue(i);
    if (Id.unwrap(id) == Id.unwrap(marketId)) {
        supplyIdx = i; // ← overwritten each time; final value = last
occurrence
        continue;
    }
    ...
}
// Rebuild loop: only skips the last occurrence; first occurrence
remains
if (supplyIdx != type(uint256).max) {
    Id[] memory newSupplyQueue = new Id[](supplyQueueLength - 1);
    for ((uint256 i, uint256 j) = (0, 0); i < supplyQueueLength; i++)
    {
        if (i == supplyIdx) continue; // ← skips last occurrence only
        newSupplyQueue[j] = IMoolahVault(vault).supplyQueue(i);
        j++;
    }
    IMoolahVault(vault).setSupplyQueue(newSupplyQueue); // ← reverts
}
```

Description

setSupplyQueue validates that every market in the new queue has a non-zero cap, but setCap(market, 0) has already executed earlier in the same transaction, making the revert inevitable:

JavaScript

```
// MoolahVault - setSupplyQueue
for (uint256 i; i < length; ++i) {
    if (config[newSupplyQueue[i]].cap == 0)
        revert ErrorsLib.UnauthorizedMarket(newSupplyQueue[i]);
    // market1.cap == 0 (setCap(0) called earlier in same tx) → revert
}
```

Impact:

Whenever the target market has duplicate entries in supplyQueue, the very first call to removeMarketFromVault fails, and every subsequent retry fails identically. The transaction reverts atomically – no funds are lost and no state is corrupted. However, market removal is completely blocked until the allocator manually calls setSupplyQueue to deduplicate the queue and the bot retries.

Skip all entries matching the target market by value rather than by index, eliminating the duplicate-entry blind spot:

Recommendation

```
JavaScript
for ((uint256 i, uint256 j) = (0, 0); i < supplyQueueLength; i++) {
  Id marketId = IMoolahVault(vault).supplyQueue(i);
  if (Id.unwrap(id) == Id.unwrap(marketId)) continue; // skip all
  matches by value
  newSupplyQueue[j] = marketId;
  j++;
}
```

Note that newSupplyQueue must be sized to the actual deduplicated length rather than supplyQueueLength - 1.

Status

Acknowledged, no fix implemented. It is presumed that admins on the MoolahVault side will never enable and add duplicate markets within the supplyQueue.

[I07] Bot can atomically finalize market removal and silently forfeit residual vault shares

Contract MoolahVaultManager.sol

Severity **Informational**

Description

Description

Relating to the fix to bug 2, in the bad debt or dust case where vaultSupplyAssets == 0 but supplyShares > 0, the previous version of removeMarketFromVault could not complete the removal. The reallocate step was skipped because it was gated on vaultSupplyAssets > 0, so the shares remained. After calling setCap(0), the subsequent updateWithdrawQueue call to drop the market reverted with InvalidMarketRemovalNonZeroSupply, since config[id].removableAt == 0 because no one had called setMarketRemoval. The market therefore stayed in the vault with its config intact, still in the withdraw queue at cap 0, and the residual shares still owned and tracked by the vault. Finalizing the removal required a CURATOR to step in and call setMarketRemoval manually, making the removal an explicit human decision.

The fix changes this, whereby the BOT can now call setMarketRemoval atomically within the removal flow, after which updateWithdrawQueue deletes config[id] and drops the market, abandoning the residual shares in the process.

Note that this increases the risk slightly as

- While the removal only applies to dead markets where the edge case scenario of `vaultSupplyAssets == 0` but `supplyShares > 0` occurs, the result is that an automated BOT role now holds the power to perform a reallocation and removal of a market from the withdrawal queue, without any human review after a bad-debt socialization.
- The CURATOR role for all MoolahVaults are held by ListaDao's Timelock contract with the address `0x2e2807F88C381Cb0CC55c808a751fC1E3fcCbb85` with a minimum delay of 1 day. As such, this fix effectively allows a BOT role to invoke

Likely no fix is required, as it is likely safe to perform atomic removal of markets given the scenario where the edge case scenario of `vaultSupplyAssets == 0` but `supplyShares > 0` would signify a worthless market

Recommendation

This finding is a note to indicate that the previous bug fix for bug 2 can actually be resolved/worked around by a manual human review on the vault side, and the slight increase in risk from allowing the BOT role to bypass the timelock when such an edge case is invoked.

Status

Acknowledged, no fix required.